Information Systems 131 (2025) 102527

Contents lists available at ScienceDirect

Information Systems

journal homepage: www.elsevier.com/locate/is

# Process-driven design of cloud data platforms

Matteo Francia *, Matteo Golfarelli, Manuele Pasini

*DISI, University of Bologna, Italy*

ARTICLE INFO

*Keywords:*
Data platform
Methodology
Big data
Cloud computing

ABSTRACT

Data platforms are state-of-the-art solutions for implementing data-driven applications and analytics. They facilitate the ingestion, storage, management, and exploitation of big data. Data platforms are built on top of complex ecosystems of services answering different data needs and requirements; such ecosystems are offered by different providers (e.g., Amazon AWS and Microsoft Azure). However, when it comes to engineering data platforms, no unifying strategy and methodology is available yet, and the design is mainly left to the expertise of practitioners in the field. Service providers simply expose a long list of interoperable and alternative engines, making it hard to select the optimal subset without a deep knowledge of the ecosystem. A more effective design approach starts with knowledge of the data transformation and exploitation processes that the platform should support. In this paper, we sketch a computer-aided design methodology and then focus on the selection of the optimal services needed to implement such processes. We show that our approach lightens the design of data platforms and enables an unbiased selection and comparison of solutions even through different service ecosystems.

## 1. Introduction

Digital transformation is one of the most disruptive trends of recent years. Digitization is perceived as the most effective solution to innovate every industrial sector thanks to process automation and the (automatic) exploitation of the value hidden in data. Such evolution is pushing information systems towards complex ecosystems of data-oriented services answering different data needs and requirements.

A data platform is a centralized infrastructure that facilitates the ingestion, storage, management, and exploitation of large volumes of heterogeneous data. It provides a collection of independent and composable services meeting the end-to-end needs of data pipelines, where: *centralized* means that a data platform is conceptually a single and unified component; *independent* means that changes in the implementation of a service do not affect other services; *composable* means that services have interfaces that enable an easy and frictionless composition; and *end-to-end* means that services cover the entire data life cycle. Data platforms foster collaboration and shared governance (being centralized, data is unified following some integration and it is easier to ensure compliance with data protection and privacy laws through shared security and access control), and scalability (being implemented on a distributed infrastructure, it is easy to add storage and computing resources as needed).

When it comes to building data platforms, no unifying strategy and methodology is there yet: building data platforms is mainly left to the expertise of practitioners or consultancy companies. On the one hand, cloud service providers[1] offer ecosystems of services composed of many engines interoperable with each other; however, choosing the optimal set of services is hard since multiple solutions could fulfill the desiderata (e.g., whether disjoint databases and data warehouses or a single Lakehouse [1] should be used) and could require vertical knowledge of the design of data pipelines. On the other hand, several abstract big data architectures have been introduced (e.g., NIST [2], Lambda [3], and Kappa [4]). However, while they provide the necessary functionalities to enable big-data applications, their implementation and adoption require to understand which services should be used.

Neither providers of service ecosystems nor abstract architectures answer a crucial question: *given an ecosystem of services and the data-driven processes to support, which is the optimal subset of services enabling such processes?* Answering this question is hard since (i) each provider offers many (overlapping) services, (ii) different providers offer different service categorizations that can hardly be mapped together, (iii) the evolution of cloud ecosystems is fast and stakeholders without vertical technical knowledge can hardly keep up with the pace, and (iv) simply selecting "all" engines is not viable due to cost and management reasons. We believe that to ease the data platform design, the

* Corresponding author.
*E-mail addresses:* m.francia@unibo.it (M. Francia), matteo.golfarelli@unibo.it (M. Golfarelli), manuele.pasini@unibo.it (M. Pasini).

[1] While data platforms are not mandatorily coupled with cloud computing, cloud computing is proving to be a winning business model since deploying and maintaining such a variety of computational resources and assets require advanced technical skills that companies hardly have "in-house".

https://doi.org/10.1016/j.is.2025.102527
Received 29 July 2024; Received in revised form 9 December 2024; Accepted 27 January 2025
Available online 4 February 2025
0306-4379/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

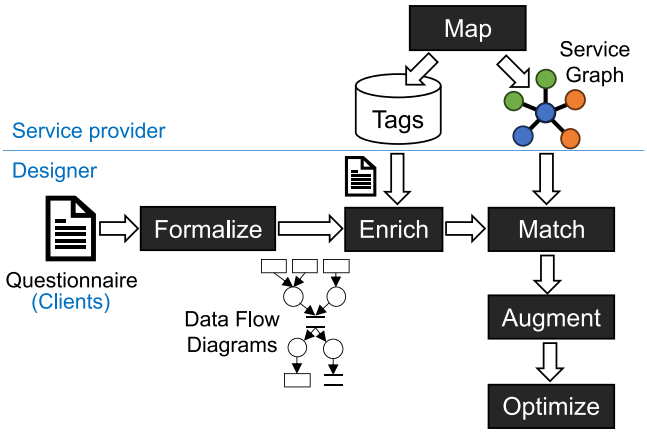


**Fig. 1.** Overview of the methodology.

description of data-driven processes should drive such activity. Indeed, data pipelines are the backbone of a *data* platform as they determine information-rich representations, and are congenial to designers since they encode many constraints on the choices to be made.

This paper introduces the following contributions.

- A methodology to guide the selection of blueprints for (cloud) data platforms driven by clients' processes.
- The formulation of such selection as an optimization problem where preferences, affinity, and further constraints can be applied.
- The discovery of (architectural) design patterns in clients' processes to align the blueprint with well-known solutions.
- Extraction of blueprints from real case studies and validation with experts from consultancy companies.

This is an extension of the methodology described in [5] and introduces these novelties: (i) management of advanced design patterns such as Lakehouse to replace both data lakes and warehouse; (ii) extension of the matching and selection steps to support and select such advanced patterns depending on their requirements; and (iii) discussion and comparison of two additional (real) case studies assessed by consultants from two Italian companies using two different service providers.

The paper is organized as follows: Section 2 sketches the overall methodology; Section 3 describes the case studies used in the paper; Sections 4 and 5 describe and formalize the design steps; Section 6 introduces the match and selection of design patterns; Section 7 evaluates the effectiveness and efficiency of the contribution; and Section 8 analyzes the related literature. Finally, Section 9 draws the conclusions and future directions.

## 2. Methodology overview

Engineering a data platform is nontrivial. A data platform must be designed upon a principle of modularity to be extensible with new components and functionalities. While it is true that reference architectures exist (as later discussed in Section 8), not all their modules and services might be necessary. Indeed, running useless functionalities and services results in a waste of resources and money (e.g., in cloud environments where users pay for the running time of the allocated resources).

To effectively build a data platform, we propose a methodology (Fig. 1) that involves three types of users: *cloud service providers* (IT experts with in-depth knowledge of services available in the ecosystem); data platform *designers* (consultants or people with expertise in designing data flows but with no vertical knowledge on cloud/service ecosystems) and *clients* asking for the design of the data platform blueprint (e.g., partners involved in the same European project with expertise on the applicative domain – such as precision agriculture – but no expertise on IT and architectures).

The methodology aims to assist designers in selecting the services necessary to implement the clients' data pipelines out of the "unstructured" lists of services provided by cloud providers.

Here, we sketch the five steps composing the methodology, and we mainly focus on steps (4), (5), and (6) in the remainder of the paper.

**(1) Define the service ecosystem.**

A cloud service provider identifies *una tantum*: (i) the alternative candidate services to compose the blueprints of data platforms, and (ii) a taxonomy of tags that describe and characterize such services. The identified services are organized in a service graph describing the preferences and relationships between them as well as the tags characterizing each service. Assuming that cloud service providers do not offer redundant services, our guideline for designing the tag taxonomy emphasizes ensuring sufficient expressiveness. This means that each service should be uniquely identifiable by a set of tags.

**Example 1.** Within the Amazon Web Services (AWS) ecosystem, there are several services for managing relational data but they have different purposes. For instance, RDS is used to manage operational workloads while Redshift is used to manage data warehouses and OLAP workloads. To distinguish these two services, the tag taxonomy should be expressive enough (i.e., have enough tags) so that RDS and Redshift differ for at least one tag.

**(2) Formalize the requirements through a Data Flow Diagram.**

When building the blueprint of a new data platform, clients compile questionnaires that collect information about their data-driven processes and the main steps, subjects, and goals of their analysis. Note that the data platform should support (possibly independent) processes from multiple clients.

Designers then refine the answers to the questionnaires and formalize the processes into a Data Flow Diagram (DFD); during the process, a few interviews with the clients might be necessary. We choose the DFD formalism [6] since it represents flows of data through an information system at a high level of abstraction, emphasizing the movement and transformation of data while hiding details such as decision points and interactions: knowing which repositories and processes compose the data-driven processes is enough to return a blueprint.

To build the DFD, designers decompose the data flows into agents, processes, and repositories. Starting from an aggregated overview, our guideline is to recursively split candidate processes and repositories until each of them is characterized by homogeneous tags (e.g., if a repository contains both unstructured images and relational tables, split it into two homogeneous repositories). Repositories and processes that should not be imported into the data platform (e.g., legacy systems deployed on-premises) should be treated as external agents.

**(3) Enrich the DFD with service tags.**

To match the DFD with the services deployed in a cloud ecosystem, the two must share the same characterization. Each process and repository in the DFD is enriched with the tags from the taxonomies previously identified by the cloud service provider. To aid designers in the characterization of each process/repository (agents are not subjects of enrichment, since they are out of the scope of the platform), clients answer an additional set of questions; we recall that clients are not required to have vertical knowledge about computer and data science or engineering. Such questions are defined by the cloud service providers and driven by the tag taxonomies. Note that since our final goal is selecting the most appropriate services, we are not interested in identifying and tracking the data/processes but rather their types and their flows.

**Example 2.** Given a repository from the DFD, we ask the question: "What are the main types of collected data?"

☐ Sensor data ☐ Images ☐ Videos ☐ Satellite observations ☐ Tables

Answering "Satellite observations" tags the repository with the properties (Volume, Big), (Data Model, File), and (Data Nature, Raster) since earth observations are data-heavy files (e.g., around 1 GB for 100 $km^2$) and tags the process to download such data as (Collection, Pull) since files are downloaded from an FTP server [7].

**(4) Match the DFD and service graphs.**

Once the DFD and service graphs are characterized by tags from the same taxonomies, it is possible to join them in a matched graph. A DFD process or repository matches (i.e., can be implemented by) a service only if the service has the same or more functionalities to fully implement it.

**(5) Augment the matched graph.**

Augmentation is a semi-automatic step atop the matched graph. On the one hand, the system automatically recognizes architectural design patterns in the data pipelines to either automatically apply or simply highlight advanced compositions of services. On the other hand, designers can manually inject their knowledge to enrich the matched graph and steer the subsequent optimization. Designers could force the selection of services in case they are required by clients, prefer some services in place of others for performance or compatibility reasons, and refine the match (e.g., consider only services supporting programming languages required by legacy software).

**(6) Select the optimal services.**

Out of all the services that are candidate implementations, it is necessary to select the minimal blueprint of the data platform that covers all the DFD entities (the fewer the services, the lower the cost and management efforts). Furthermore, dependencies and compatibilities between services have to be taken into account.

## 3. Case study: Agritech

Within the Agritech spoke of the NRRP European project [8], we deploy a data platform supporting prescriptive analytic tasks from 8 clients in the field of precision agriculture. We will use one of the partners as a working case study throughout the paper. Here follows a qualitative description of one of the clients' data flows that the platform must support.

**Example 3.** The analysis entails a flow that is structured as follows. (i) Data comes from soil moisture sensor grids, weather stations, and SENTINEL-2 satellites; (ii) Sensor and weather data is uploaded every 15 min to the platform, while satellite data is periodically downloaded; (iii) Soil moisture data is interpolated using mathematical (e.g., bilinear interpolation) and machine learning (e.g., neural networks) techniques; (iv) Vegetation indexes are computed out of the raw satellite observations and integrated with the enriched sensor data; (v) Reports are periodically generated out of the integrated data; (vi) Given an optimal soil moisture matrix, the integrated data is used to decide how much to irrigate the soil.

After gathering questionnaires from the clients, we (designers) iteratively refined the interview into a DFD.

**Example 4** (*DFD*)**.** Fig. 2 depicts the tasks from Example 3 using the DFD formalism.

- Moisture Sensors and Weather Stations stream data into the platform, such data is Consumed and stored in the original format into Sensor Data;
- Satellite images are periodically Downloaded and stored into Raw Images;

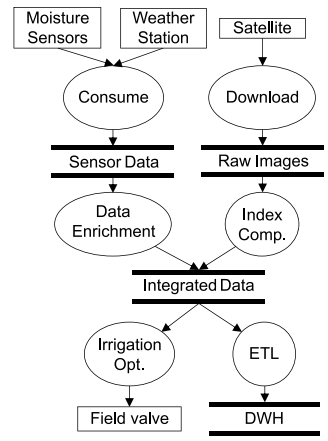


**Fig. 2.** DFD from Example 3.

- Sensor Data are Enriched, interpolated, and stored into Integrated Data;
- Raw Images are used for the computation of vegetation indexes and integrated with sensor data;
- Integrated Data are used to fuel a Data Warehouse through ETL;
- The Irrigation Optimization algorithm controls the Field Valves installed in Emilia Romagna, Italy.

Following the guidelines, the data collection processes cannot be merged into a single one since (i) Moisture Sensors streams data into the platform while Satellite images are periodically downloaded, and (ii) Sensor Data and Raw Images cannot be grouped into a single repository since they contain heterogeneous data types. More details on such tasks are described in [9,10].

## 4. Mapping the service ecosystem

Tags (Table 1) are organized as a collection of hierarchies that can be fueled bottom-up from the documentation of cloud services providers (i.e., the set of tags that are attached to each service or that are inferrable through the service description), and top-down from the literature; for instance, the big data V's [11], such as volume (small to big), variety (structured to unstructured), and velocity (low to high).

**Definition 1** (*Hierarchy*)**.** A *hierarchy* $h$ is a taxonomy of categorical values $Dom(h)$. $\geq_h$ is the partial order that defines the taxonomy; $\geq_h$ indicates whether a value is more specific than another. We denote with $H$ the set of hierarchies.

**Example 5** (*Tags and Hierarchies*)**.** Table 1 depicts an example of hierarchies $H = \{\mathsf{DataModel}, \mathsf{Volume}, \ldots\}$. In the partial order of the hierarchy $h = \mathsf{DataModel}$ it is, for instance, Relational $\geq_{\mathsf{DataModel}}$ Structured $\geq_{\mathsf{DataModel}}$ Data Model (All). Relational is more specific than Structured in the Data Model hierarchy.

The services available on a specific ecosystem differ for each provider, and no single and shared organization of services is available. Table 2 shows an example of services from AWS (as of November 2023).

While a plethora of services is available, there is no necessity to consider all of them at once. Indeed, the considered services must cover the "core" functionalities necessary to run a data platform such as the ones proposed by NIST [2], among them storage and data processing engines. Also, it is necessary to consider the relationships between services (e.g., whether the adoption of a service requires another) and preferences in their choice (e.g., due to differences in performance reasons). That is why we map the services into a directed property graph.

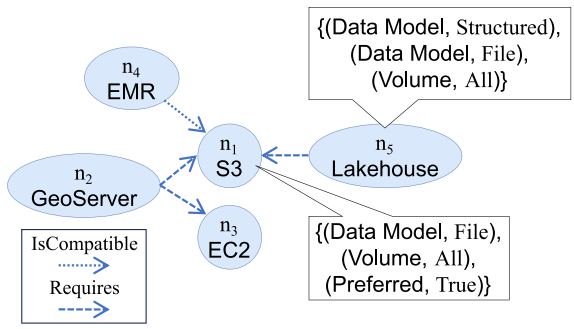


**Fig. 3.** Excerpt of a service graph and its properties.

**Definition 2** (*Directed Property Graph*)**.** A *directed property graph* is a tuple $G = (N, A, P, L)$ where $N = \{..., n_i, ...\}$ is a finite set of *nodes*, $A = \{..., a_{ij}, ...\}$ is a finite set of *arcs* connecting nodes $n_i$ and $n_j$, $P = \{..., (h, v), ...\}$ is a set of key–value *properties* with $h \in H$ and $v \in Dom(h)$, and $L$ is a set of *labels*. $props : (N \cup A) \rightarrow P$ returns the properties of a node or arc. $label : (N \cup A) \rightarrow L$ returns the labels of a node or arc.

**Definition 3** (*Path*)**.** Given a directed property graph $G$, a *path* $p_{ij}$ from node $n_i$ to node $n_j$ is a sequence of distinct arcs $(a^1_{ix}, \ldots, a^k_{xy}, \ldots, a^{|p|}_{yj})$ for which there is a sequence of distinct nodes $(n^1_i, \ldots, n^k_x, n^{k+1}_y, \ldots, n^{|p|+1}_j)$ such that the arc $a^k$ connects the node $n^k$ to the node $n^{k+1}$ for $k \in [1, |p|]$. We denote with $p_{ij}.nodes$ the sequence of nodes connected by the path.

**Definition 4** (*Service Graph*)**.** A *service graph* is a directed property graph $G^S$ where nodes are labeled as Service, while arcs are alternatively labeled as $\{\text{Requires}, \text{IsCompatible}, \text{IsAkin}\}$.

The semantics of the labels is the following:

- Service: is any engine from the service ecosystem that can be optionally tagged with the property (Preferred, True) to specify whether it should be considered more than others;
- Requires: represents whether a service mandatorily relies on another;
- IsCompatible: represents whether a service natively interfaces with another (i.e., their interaction is supported by default and does not require custom/additional libraries or connectors);
- IsAkin: represents whether a service not only is compatible with another but also if their adoption together is encouraged.

**Example 6** (*Service Graph*)**.** Here follows an excerpt of the service graph $G^S$ and properties built out of Table 2 and depicted in Fig. 3.

$N = \{n_1, n_2, n_3, n_4, n_5, \ldots\}, \; A = \{a_{23}, a_{34}, \ldots\}$

$n_1 = \texttt{S3}, \; n_2 = \texttt{GeoServer}, \; n_3 = \texttt{EC2}, \; n_4 = \texttt{EMR}, \; n_5 = \texttt{Lakehouse}$

$label(n_1) = \text{Service}$

$label(a_{23}) = \text{Requires}$

$label(a_{41}) = \text{IsCompatible}$

$props(n_1) = \{(\text{Data Model}, \text{File}), (\text{Volume}, \text{All}), (\text{Preferred}, \text{True})\}$

$props(n_2) = \{(\text{Data Model}, \text{File}), (\text{Data Nature}, \text{Raster})\}$

$props(n_5) = \{(\text{Data Model}, \text{Structured}), (\text{Data Model}, \text{File}), (\text{Volume}, \text{All})\}$

`GeoServer` requires `EC2` since it is deployed on it, `EMR` is compatible with `S3` since it can natively read from and write to the object storage.

**Table 1**
Examples of taxonomies of tags.

| | | |
|---|---|---|
| Data Model (All) | Structured | Relational |
| | | Multidimensional |
| | Semi-structured | Document |
| | | Wide-column |
| | Unstructured | Key–value |
| | | Graph |
| | | File (text, binary) |
| Data Nature (All) | Spatial | Vectorial |
| | | Raster |
| | Temporal | |
| Volume (All) | Small | |
| | Big | |
| Data Zones (All) | Landing | |
| | Archive | |
| | Processed | |
| Functionality (All) | ETL | |
| | Reporting | |
| | Operational | |
| | Machine learning | Classification |
| | | Regression |
| Collection (All) | Pull | Cumulative |
| | | Delta |
| | Push | |
| Computing (All) | Batch | |
| | Mini-batch | |
| | Streaming | |
| Language (All) | Low code | |
| | Python | |
| | Java | |
| | SQL | |

## 5. Process-driven match, augmentation, and optimization

We now describe three steps composing our approach. *Match* extracts the services that can support the execution of the data pipeline, *Augment* refines the potential candidates and, finally, *Optimize* picks the optimal (minimal) subset. We mainly focus on the optimization of the platform design rather than on how, starting from questionnaires, designers refine the DFD of such flows.

Since data pipelines are the backbone of data platforms, everything starts from the DFD describing the processes of clients' analysis.

**Definition 5** (*Data Flow Diagram*)**.** A *Data Flow Diagram* (DFD) is a directed property graph $G^D$ where nodes are alternatively labeled as $\{\text{Agent}, \text{Repository}, \text{Process}\}$, while arcs are labeled as Flow. Arcs must connect with at least one node with label Process. The semantics of the labels is the following:

- Agent: an external entity that communicates with the system and stands outside of the system;
- Process: transform inputs to outputs;
- Repository: store data for later use;
- Flow: transfer data from one part of the system to another.

**Table 2**
An excerpt of services from the Amazon AWS ecosystem (from https://aws.amazon.com/big-data/datalakes-and-analytics).

| Solution areas | Use cases | AWS services |
|---|---|---|
| Advanced analytics | Interactive analytics | Athena |
| | Big data processing | EMR |
| | Data warehousing | Redshift |
| | Real-time analytics | Managed Service for Apache Flink |
| | Operational analytics | OpenSearch Service |
| | Dashboards and visualizations | QuickSight |
| | Visual data preparation | Glue DataBrew |
| Data management | Real-time data movement | Managed Streaming for Apache Kafka, Kinesis Data Streams, Glue |
| | Data governance | DataZone, Glue, Entity Resolution, Lake Formation, S3 |
| | Object storage for data lakes | S3, Lake Formation |
| | Backup and archive for data lakes | S3 Glacier, Backup |
| | Data catalog | Glue, Lake Formation |
| | Third-party data | Data Exchange, Clean Rooms |
| Machine learning | Frameworks and interfaces | Deep Learning AMIs |
| | Platform services | SageMaker |

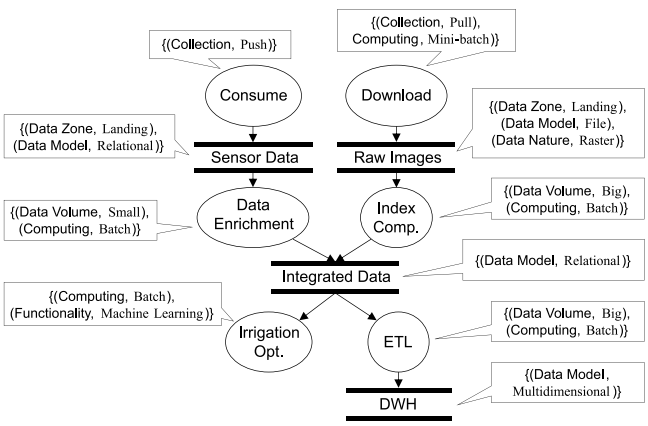


**Fig. 4.** An excerpt of properties of the DFD from Fig. 2.

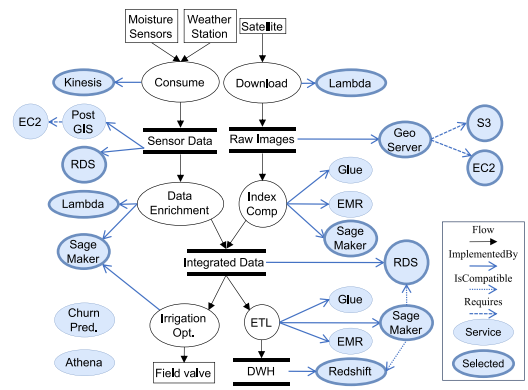


**Fig. 5.** Matching the DFD (white) and service (blue) graphs; in bold the services selected as the optimal blueprint for the data platform design. `Athena` and `Churn Prediction` are examples of unmatched services that are available in the AWS ecosystem. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

We recall that the DFD is enriched with tags identified by the cloud service provider (Table 1).

**Example 7** (*DFD*)**.** Here follows an excerpt of the DFD $G^D$ and properties from Fig. 4.

$N = \{n_1, n_2, n_3, \ldots\},\ A = \{a_{12}, \ldots\}$

$n_1 = \text{Consume},\ n_2 = \text{Sensor Data},\ n_3 = \text{Raw Images}$

$label(n_1) = \text{Process}$

$label(n_2) = \text{Repository}$

$label(n_3) = \text{Repository}$

$label(a_{12}) = \text{Flow}$

$props(n_2) = \{(\text{Data Model}, \text{Relational})\}$

$props(n_3) = \{(\text{Data Model}, \text{File}), (\text{Data Nature}, \text{Raster})\}$

### 5.1. Matching the DFD and service graphs

Given the service graph and the DFD, we can automatically join them by matching their tags.

**Definition 6** (*Match*)**.** Given a DFD and a service graph, the function $match : N^D \times N^S \to \{true, false\}$ is true when, given a node $n^D \in N^D$ of the DFD and a node $n^S \in N^s$ of the service graph, each property in $props(n^D)$ matches a property in $props(n^S)$. A property $(h_i, v_i)$ matches another property $(h_j, v_j)$ if $h_i = h_j$ and $v_i \geq_{h_i} v_j$.

A match represents whether a DFD node can be implemented using a specific service (solid arcs in Fig. 5); i.e., services that have the same or more generic values for the properties specified in the DFD node. For instance, a DFD repository tagged as Relational could be implemented by services supporting Relational or all Structured data.

**Example 8** (*Match*)**.** With reference to Fig. 5, given the node Raw Images from the DFD, and the nodes `S3` and `GeoServer` from the service graph

$$props(\text{Raw Images}) = \{(\text{Data Model}, \text{File}), (\text{Data Nature}, \text{Raster})\}$$

$$props(\texttt{GeoServer}) = \{(\text{Data Model}, \text{File}), (\text{Data Nature}, \text{Raster})\}$$

$$props(\texttt{S3}) = \{(\text{Data Model}, \text{File}), (\text{Volume}, \text{All})\}$$

Since $match(\text{Raw Images}, \texttt{GeoServer}) = true$ and $match(\text{Raw Images}, \texttt{S3}) = false$, Raw Images can be implemented by `GeoServer` but not directly in `S3`. The former is natively capable of managing geographical data while `S3` would only provide storage as plain files.

**Definition 7** (*Matched Graph*)**.** Given a DFD $G^D = (N^D, A^D, P^D, L^D)$ and a service graph $G^S = (N^S, A^S, P^S, L^S)$, a matched graph $G^M = (N^D \cup N^S, A^D \cup A^S \cup A, P^D \cup P^S, L^D \cup L^S \cup \{\text{ImplementedBy}\})$ is a directed property graph obtained as the union of $G^S$ and $G^D$. $A$ is an additional set of arcs, one for every match between nodes in $N^S$ and $N^D$. Arcs in $A$ are labeled as ImplementedBy and can have an optional property (Mandatory, True) to specify that such implementation must be selected in the final blueprint.

The result of a match is a graph that is composed of the union of the nodes, and the union of the arcs plus additional arcs that represent candidate implementations for the DFD processes/repositories. The label ImplementedBy represents whether a DFD process or repository can be implemented by a specific service. If no match is found for a DFD node, we alert the designer in order to verify that the DFD has been properly designed and/or enriched with the service tags.

**Example 9** (*Matched Graph*)**.** Fig. 5 depicts an excerpt of the matched graphs for the DFD from Fig. 2; for the sake of clarity, not all the arcs have been presented. Nodes from the DFD have a white background, while services are represented in blue. Solid arrows represent arcs labeled as ImplementedBy (e.g., Sensor Data can be implemented by either `PostGIS` or `RDS`). Dotted arrows represent arcs labeled as IsCompatible (e.g., `SageMaker` reads from and writes to `Redshift`). Dashed arrows represent arcs labeled as Require (e.g., `GeoServer` requires `EC2` since it is deployed on it). Of course, it is unlikely that all services of the cloud ecosystem will be matched, `Athena` and `Churn Prediction` are examples of unmatched services.

### 5.2. Augmenting and pruning the matched graph

Designing a platform is not just a technical task but encompasses various organizational, business, and political aspects that must be carefully considered. The successful creation of a platform requires the expertise and experience of designers to manipulate the matched graph and aid in the subsequent optimization. This ensures that the platform is not only technically sound but also aligns with broader strategic goals and user needs.

In our approach, designers have several tools at their disposal to influence and refine the design process:

- Force the selection of specific services: designers can mandate the inclusion of certain services due to various factors such as client requirements, industry standards, or established design practices. This is accomplished by adding the property (Mandatory, True) to ImplementedBy arcs in the matched graph. By doing so, designers ensure that essential services are incorporated into the platform.
- Prefer certain services over others: in many cases, some services are more desirable than others due to their superior efficiency, reliability, or user familiarity. Designers can indicate these preferences by adding the property (Preferred, True) to the nodes labeled as Service in the matched graph. This preference can optimize the platform's blueprint and user experience by prioritizing the most advantageous services.
- Refine the match: designers can fine-tune the matched graph by preliminarily removing services that do not meet specific criteria or adding services that were initially unmatched. For example, they might exclude services that do not support required programming languages for legacy software or add missing matches due to incomplete tagging. This refinement ensures the platform is both relevant and comprehensive, accommodating all necessary components and integrations.

**Example 10** (*Mandatory and Preferred Services*)**.** With reference to Fig. 5, imagining that our methodology is used to lift-and-shift an on-premises solution that already used `GeoServer` to AWS, `GeoServer` could be tagged with the property (Mandatory, True) since clients already have experience with it (and not with alternative engines such as `GoogleEarth`). Also, since `S3` is cheaper than other storage services (such as `EBS` and `EFS`), `S3` could be tagged with the property (Preferred, True).

After manual adjustments, an automated pruning process removes sub-graphs that are not reachable from the DFD. These sub-graphs represent services that are neither candidate implementations nor accessible from candidate implementations.

**Example 11** (*Pruning the Matched Graph*)**.** In Fig. 5, `Churn Prediction` and `Athena` are discarded before the optimization since they are not reachable from any DFD entity (i.e., they are neither candidate implementations nor required by other services).

Finally, this step supports the recognition, matching, and refinement of design patterns, as discussed in Section 6. Design patterns ensure that the data platforms adhere to best practices and maintain a high standard of quality and maintainability.

### 5.3. Optimizing the blueprint

The optimization of the services comprising a data platform can take various forms, depending on the numerous influencing factors and the diverse objectives of clients, such as cost reduction, simplified management, or enhanced extensibility. In this paper, we use the minimization of the number of services as an objective function since this indicator correlates with both cost minimization and management simplicity. It will be seen later that other desirable properties of the chosen solution (e.g., adherence to business practices and compatibility with other systems already present in the company) can be easily satisfied through the preferences indicated by users.

However, such optimization is not trivial. For instance, some DFD entities can be implemented atop alternative services (e.g., a data lake that is implemented either on `S3` or `HDFS`), and some services are meaningful only if they cover at least two DFD entities (e.g., we use a `Lakehouse` to replace both a data lake and a data warehouse). The selection must be compliant with the following conditions.

- *Coverage*: all processes and repositories in the DFD must be covered.
- *Dependency*: if a service is selected, all its required services must be (recursively) selected too (e.g., `Geoserver` is deployed on `EC2`).
- *Compatibility*: a service can be selected only if it is compatible with the services selected for the previous (if existing) and following (if existing) nodes in the DFD.
- *Preference*: services marked as preferred should have more chances to be selected (for instance, preferences can be expressed for services entailing lower costs or higher performance).
- *Mandatory*: service implementations marked as mandatory must be selected for the blueprint (e.g., a certain repository must be implemented by a document-based service).
- *Affinity*: if a service is selected, services with high affinity should be selected too.

More formally, given a matched graph $G^M = (N, A, P, L)$, the selection of the optimal blueprint can be modeled as a linear programming problem inspired by the standard *facility location problem*. In a basic formulation, the facility location problem consists of a set of potential facility sites where a facility can be opened, and a set of demand points that must be serviced. The goal is to pick a subset of facilities to open while minimizing the sum of distances from each demand point to its nearest facility plus the sum of opening costs of the facilities. In our case, services are mapped to the potential locations while DFD nodes are mapped to the demand points. The formulation in Fig. 6 reads as follows.

(1) The optimization function minimizes the weighted sum of the selected services. Weights $w_i$ specify preferences for services $s_i$, we set

$$min \sum_i w_i s_i \text{ such that} \tag{1}$$

$$s_i \in \{0,1\} \text{ for all } n_i \in N, label(n_i) = \text{Service} \tag{2}$$

$$s_{ij} \in \{0,1\} \text{ for all } a_{ij} \in A, label(a_{ij}) = \text{ImplementedBy} \tag{3}$$

$$s_j \geq s_{ij} \text{ for all } s_{ij} \tag{4}$$

$$\sum_{a_{ij} \in A, label(a_{ij}) = \text{ImplementedBy}} s_{ij} = 1 \text{ for all } n_i \in N, label(n_i) \in \{\text{Process}, \text{Repository}\} \tag{5}$$

$$s_j \geq s_i \text{ for all } a_{ij} \in A, label(a_{ij}) = \text{Requires} \tag{6}$$

$$s_{ij} + s_{kh} \leq 1 \text{ for all } a_{ik} \in A, a_{jh} \notin A, label(a_{ik}) = \text{Flow}, label(a_{jh}) = \text{IsCompatible} \tag{7}$$

$$s_{ij} = 1 \text{ for all } a_{ij} \in A, (\text{Mandatory}, \text{True}) \in props(a_{ij}) \tag{8}$$

$$s_{ij} \cdot s_{kh} \geq s_h \text{ for all } a_{ik}, a_{jh} \in A, label(a_{ik}) = \text{Flow}, label(a_{jh}) = \text{IsAkin} \tag{9}$$

**Fig. 6.** Selecting the optimal services considering coverage (5), dependency (6), compatibility (7), and mandatory (8), and affinity (9) constraints.

$$w_i = \begin{cases} 0.5 \text{ if } (\text{Preferred}, \text{True}) \in props(n_i) \\ 1.0 \text{ otherwise} \end{cases}$$

In case multiple optimal solutions exist, we return all of them to the designers so that they can compare them and select the one they find the most appropriate since there is no solution dominating the others. When no solution is feasible (e.g., due to a lack of matches or incompatibility of mandatory services) the system alerts the designers since a refinement of the DFD could be necessary.

(2) $s_i$ are binary variables modeling services. $s_i = 1$ if the service is selected, 0 otherwise.

(3) $s_{ij}$ are binary variables modeling the services implementing DFD processes and repositories. $s_{ij} = 1$ if $n_i \in N$ s.t. $label(n_i) \in \{\text{Repository}, \text{Process}\}$ is implemented by $n_j \in N$ s.t. $label(n_j) = \text{Service}$.

(4) Binding variables $s_j$ and $s_{ij}$: service $s_j$ is selected if it implements ($s_{ij} = 1$) a DFD node $n_i \in N$ s.t. $label(n_i) \in \{\text{Repository}, \text{Process}\}$.

(5) *Coverage* constraints: every repository and process must be implemented by exactly one service.

(6) *Dependency* constraints: if a service is selected, all the services it depends on must be selected.

(7) *Compatibility* constraints: if two services are incompatible, they cannot be selected to implement two consecutive processes/repositories linked through a data flow in the DFM.

(8) *Mandatory* constraints: if an implementation is mandatory and tagged with the property (Mandatory, True), $s_{ij}$ is set to 1.

(9) *Affinity* constraints: if the service $s_h$ is selected and has affinity with another service $s_j$, then $s_j$ is selected too.

**Example 12** (*Optimization*)**.** Given the matched graph from Fig. 5, the *optimal* blueprint of the data platform is represented by the services highlighted in bold (with a cost of 7.5), namely `Kinesis`, `Lambda`, `RDS`, `EC2`, `SageMaker`, `GeoServer`, `S3` (tagged as preferred), and `Redshift`. `GeoServer` has been selected since it is the only available implementation for Raw Images, `EC2` and `S3` have been selected since they are required by `GeoServer`. `SageMaker` has been selected and preferred to `EMR` and `Glue` since three DFD processes can be implemented on it.

## 6. Design patterns

Clients' data pipelines could present *design patterns* (i.e., solutions to common problems in software and architecture design) that require certain constraints to be satisfied on the overall data pipeline (and not only on single nodes in the DFD and their properties). Design patterns are important since they impact the services that are matched and selected for the optimal blueprint.

### 6.1. Defining design patterns

Design patterns are retrieved from the DFD graph.

**Definition 8** (*Architectural Design Pattern*)**.** Given a DFD graph $G^D$, a design pattern $p$ is a subgraph $\bar{G} \in G^D$ in which $cond_p(\bar{G})$ holds. $cond_p(\bar{G})$ is specific to each design pattern and can be expressed in terms of graph queries on the DFD graph.

Since our definition of design pattern is "generic", design patterns could impact our approach in different ways. The simple recognition of design patterns could already be informative for the designer of the data platform (the recognition of the pattern itself could help to refine the design of the platform). Additionally, patterns could alter the matched graph (by matching additional services) and the optimization procedure (by preferring some services to others). We show the impact of design patterns with reference to the *Lakehouse* and *Lambda Architecture* (although other patterns exist, such as the kappa architecture and the 2- or 3-layered architecture for data warehousing).

*Lakehouse.* Lakehouse (Fig. 7) is a data management paradigm that combines the capabilities of data lakes and data warehouses, enabling business intelligence and machine learning on the same data. A Lakehouse can be applied if, within the same data pipeline, there is a sequence of ETL steps that moves data from a data lake to a data warehouse, optionally passing through relational databases. Lakehouse encourages the adoption of a medallion architecture (a data design pattern)[2] where raw data is initially stored in a data lake (bronze layer), refined data are stored in a relational engine (silver layer), and data cubes are stored in a multidimensional engine (gold layer). However, to avoid the additional usage of a relational engine (which would bring higher costs and effort into keeping the 3 layers aligned), ETL could skip the intermediate storage and upload data directly into the data warehouse for simpler architectures.

Formally, a Lakehouse is a graph $\bar{G}$ with a set of nodes $\bar{N} = \{n_i, \ldots, n_k, \ldots, n_j\}$ such that $cond_{Lakehouse}(\bar{G})$ is defined as

$$\exists p_{ij} \ s.t. \ label(n_i) = Repository \wedge (\text{Functionality}, \text{Landing}) \in prop(n_i) \wedge$$
$$label(n_j) = Repository \wedge (\text{Data Model}, \text{Multidimensional}) \in prop(n_j)$$

and (optionally) $n_k \in p_{ij}.nodes \ s.t.$ (Data Model, Structured) $\in prop(n_k)$. Intuitively, $n_i$ is the data lake, $n_j$ the data warehouse, and $n_k$ the optional intermediate relational storage.

[2] https://www.databricks.com/glossary/medallion-architecture (last access 2024-12-01).

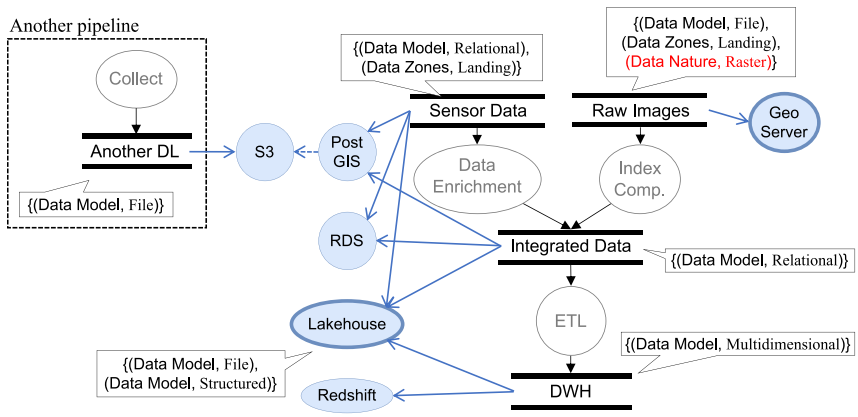


**Fig. 7.** Matching the Lakehouse pattern; in gray the processes that are not relevant for matching the Lakehouse pattern and in red the property preventing the match between Raw Images and `Lakehouse`.

*Lambda architecture.* Lambda architecture (Fig. 8) is a data-processing architecture designed to handle massive quantities of data by taking advantage of both batch and stream processing. It uses batch processing to provide comprehensive and accurate views of batch data, while simultaneously using real-time stream processing to provide views of online data. Formally, a Lambda architecture is a graph $\bar{G}$ with a set of nodes $\bar{N} = \{n_i, n_j, n_k, n_l\}$ such that $cond_{Lambda}(\bar{G})$ is defined as the conjunction of the following conditions

$$\exists p_{ij} \ s.t. \ label(n_i) = Process \wedge (\text{Collection}, \text{Push}) \in prop(n_i) \wedge$$
$$label(n_j) = Process \wedge (\text{Computing}, \text{Streaming}) \in prop(n_j),$$
$$\exists p_{ik} \ s.t. \ label(n_k) = Repository \wedge (\text{Volume}, \text{Big}) \in prop(n_k),$$
$$\exists p_{kl} \ s.t. \ label(n_l) = Process \wedge (\text{Computing}, \text{Batch}) \in prop(n_l)$$

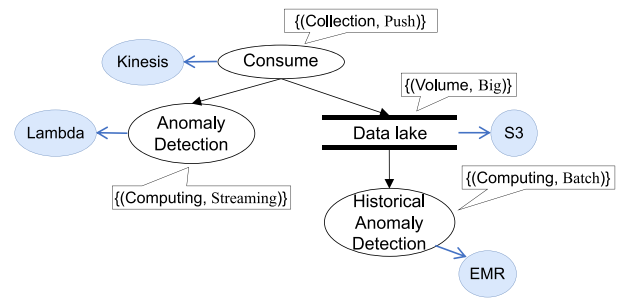


**Fig. 8.** Lambda pattern.

### 6.2. Exploiting the matched graph

Design patterns can be highlighted to designers for further evaluation or match additional services. For instance, new matches are added when design patterns can be directly implemented by services from the service graph. This is the case for Lakehouse, where a single service fulfills the requirements of an entire pattern.

*Lakehouse.* Databricks' `Lakehouse` is a multi-platform service that implements the Lakehouse pattern. Given a DFD graph including one (or more) Lakehouse pattern $\bar{G}$ with nodes $\bar{N}$ and a service graph including the node $n_s = \texttt{Lakehouse}$, the matched graph $G^M$ is augmented with new arcs $(N^M, A^M \cup A, P^M, L^M)$ where $A = \{a_{is} \ s.t. \ match(n_i, n_s) = true, n_i \in \bar{N}\}$ is an additional set of arcs, one for every (property) match between nodes in $\bar{N}$ and $n_s$. Arcs in $A$ are labeled as ImplementedBy.

In other words, the Lakehouse is a potential implementation for the DFD repositories in the path connecting a data lake to a data warehouse.

**Example 13** (*Design Pattern Match*). Fig. 7 depicts two examples of matching the service `Lakehouse` (processes are grayed out since they are not relevant for matching the Lakehouse pattern). `Lakehouse` does not match Another DL since the latter belongs to a pipeline without a data warehouse. Conversely, `Lakehouse` matches Sensor Data, Integrated Data, and DWH since in the DFD graph exists a path connecting a landing area (Sensor Data) to a data warehouse (DWH) and optionally passing through a relational repository (Integrated Data). In other words, this data pipeline is organized according to the medallion architecture that can be implemented by `Lakehouse`. Additionally, `Lakehouse` does not match Raw Images since it has the property (Data Nature, Raster) and `Lakehouse` cannot directly manage spatial data/queries (more formally $match(\text{Raw Images}, \texttt{Lakehouse}) = false$).

Noticeably, if `Lakehouse` is matched and later selected in place of RDS and `Redshift`, the cost of the solution in Example 12 and Fig. 5 decreases from 7.5 to 6.5.

*Lambda architecture.* The lambda architecture does not require any augmentation since the composition of the services necessary for its implementation is already matched through the properties of single nodes. However, as we will see immediately below, its exploitation is accomplished by a modified formulation of the optimization problem.

### 6.3. Refining the optimization

To implement the design patterns, additional constraints could be plugged into the optimization or the weights of some services could be adjusted to prefer the adoption of some services to others.

*Lakehouse.* The adoption of the Lakehouse pattern $\bar{G}$ with nodes $\bar{N} = \{n_i, \ldots, n_k, \ldots, n_j\}$ (with $n_i$ representing a data lake, $n_j$ representing a data warehouse, and $n_k$ is an optional structured repository in between the data lake and the data warehouse) requires that, if `Lakehouse` service $n_s$ is picked (i.e., $s_s = 1$), such service must cover the data lake and the data warehouse as well as the optional intermediate structured repositories (there is no point in having a partial implementation of a `Lakehouse`).

$$s_{is} \cdot \ldots \cdot s_{ks} \cdot \ldots \cdot s_{js} > s_s \text{ for all } a_{is}, a_{ks}, a_{js} \in A,$$
$$label(a_{is}) = label(a_{ks}) = label(a_{js}) = \text{ImplementedBy}$$

*Lambda architecture.* The adoption of the Lambda architecture $\bar{G}$ with nodes $\bar{N} = \{n_i, n_j, n_k, n_l\}$ (with $n_i$ representing a streaming source, $n_j$ a streaming processing, $n_k$ a big data storage, and $n_l$ a big data engine) does not require additional constraints but pushes towards

the adoption of some services. For instance, the AWS documentation usually leverages S3 as a big data storage, EMR as a big data engine, Kinesis as a streaming source, and Lambda as streaming processing (although different services could be used, such as queuing services in place of Kinesis). Since these services are *recommended* by AWS to create a Lambda Architecture, their properties are extended with the additional property (Preferred, True).

$$props(\texttt{S3}) = \{(\textsf{Preferred}, \textsf{True})\} \cup props(\texttt{S3})$$

$$props(\texttt{EMR}) = \{(\textsf{Preferred}, \textsf{True})\} \cup props(\texttt{EMR})$$

$$props(\texttt{Kinesis}) = \{(\textsf{Preferred}, \textsf{True})\} \cup props(\texttt{Kinesis})$$

$$props(\texttt{Lambda}) = (\textsf{Preferred}, \textsf{True}) \cup props(\texttt{Lambda})$$

Given the optimization problem formulated in Section 5.3, from Line (1) it follows that $w_i = 0.5$ if $s_i \in \{\texttt{S3}, \texttt{EMR}, \texttt{Kinesis}, \texttt{Lambda}\}$ since $w_i$ is set to 0.5 if (Preferred, True) $\in props(s_i)$. In other words, weights are decreased to 0.5 to bias the optimization towards the selection of the four services.

## 7. Evaluation

We implemented our approach (available at https://github.com/big-unibo/DataPlatformDesign) using GraphDB (to store the taxonomy of tags, the service graph, and the DFD diagrams) and Python, where we leverage the CPlex library to solve the optimization problem.

### 7.1. Validation with real use cases

To assess the approach, we applied it in two real contexts[3]: Company1 (one of the largest Italian consulting firms specialized in data analysis and with strong expertise in the field of data platforms and cloud computing) and Company2 (an international company leading in the production of smart and digitalized sports equipment). In particular, (i) we asked their experts for two use cases of data platforms implemented on different cloud service providers to prove the portability of our approach, (ii) we generated a blueprint for each use case, (iii) we asked the experts to evaluate and compare our blueprints against the ones they implemented.

In this experimental setup, we act as platform designers, whereas experts from the companies act as clients. The collection of the case studies was done by setting up 2 to 3 interviews of a maximum of 1 h each and according to the methodology from Section 2.

- Step (1): we created the service graphs for Microsoft Azure and Google Cloud. We adopted the taxonomy of tags summarized in Table 1.
- Step (2): the first interview was devoted to understanding the proposed data-driven processes: the main steps, subjects, and analysis goals. The output of the interview was the DFD diagram of the processes.
- Steps (3) and (4): we enriched the DFD with tags from our taxonomy and matched it with the service graphs.
- Step (5): the second interview was devoted to refining the DFD and augmenting the matched graph with the clients. This additional iteration on the DFD is necessary in case of ambiguities in the tagging.
- Step (6): we generated the blueprint and used the final interview to discuss the solution with the clients.

Here follows a description of the Case Studies (CS). Their DFDs along with their blueprints are represented in Figs. 9 and 10.

[3] We anonymize the names of the companies due to non-disclosure agreements.

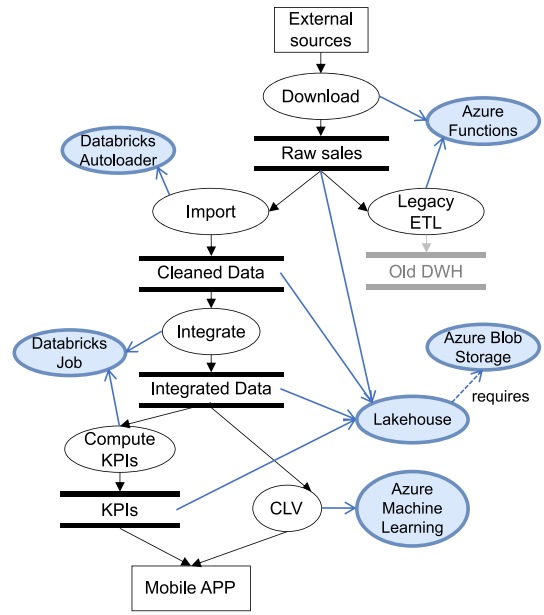


**Fig. 9.** Matched graph and selected services for case study #1.

**Example 14** (*CS#1: Creating a KPI Factory*)**.** Starting from an on-premises data warehouse that has reached a performance plateau, a sales company aims to create a KPI factory: a centralized system where all KPIs are incrementally computed. The data warehouse only supports KPI calculations four times a day, but the company needs more frequent updates. The KPI calculation starts with sales data being drawn from external sources every hour via HTTP API. This data is then stored in its original format, cleaned, integrated, and made usable on a multi-dimensional engine. It is crucial to store the data at each processing step to facilitate different levels of analysis. For instance, the integrated data is utilized for machine learning models to estimate customer lifetime value. The calculated KPIs are sent to applications managed by client advisors for client-telling actions and are accessible via JDBC connectors. Additionally, the (old) data warehouse must continue to be updated with new data through its legacy ETL procedure. Since the on-premises data warehouse is out of the scope of the platform, we grayed it out in Fig. 9.

**Example 15** (*CS#2: Forecasting Customer Lifetime Value*)**.** From a Salesforce CRM ecosystem, users' data is extracted every thirty minutes via API calls leveraging a change-data-capture mechanism and is stored in .csv files. Daily, such .csv files are then pushed into two buckets: the first acts as an ingestion backup storage while the other serves as a staging area. From the latter on, data flows through an ETL procedure that updates both the historical trend and the real-time view of each user. The output of the ETL procedure is transformed through feature engineering and fed to a machine learning model predicting the customer lifetime value. Finally, inferred data is filtered and copied into the Salesforce CRM via a custom application.

All in all, we draw the following conclusions.

- **Blueprints effectiveness.** The blueprints returned by our system are compatible and overlap with the ones actually adopted by the companies. In CS#1, our solution matches the company's solution with 6 services out of 7. The only difference is Azure Machine Learning. The company preferred to keep the machine learning activity within the Databricks ecosystem rather than in Azure. This could be fixed by imposing a mandatory service to fulfill the CLV process. Similarly, in CS#2 we matched 4 services out

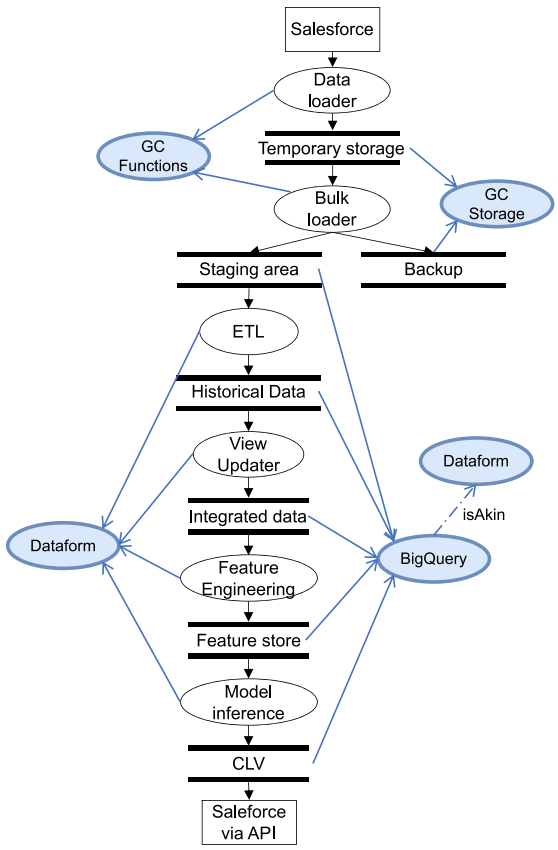


**Fig. 10.** Matched graph and selected services for case study #2.

of 5 since we adopted `Google Cloud Functions` for the Data Loader process but the company preferred to use `GSUtils` within the `Google Cloud CLI` (command line interface). `BigQuery` is akin to `Dataform` since `Dataform` operationalizes scalable data transformations pipelines in `BigQuery` using SQL [12].

- **Methodology effectiveness.** The requirement analysis was smooth and the staff from the two companies had no problem describing the data-processes and answering questions needed for tagging. The case study from Company1 required a refinement of the DFD since it involved several organizational constraints. For instance, the Raw Sales repository is handled by a different department than the one handling the repositories Cleaned Data, Integrated Data, and KPIs. This organizational constraint impacts the blueprint. On the one hand, if the two departments were willing to share the same technological solution, then all repositories could be implemented by `Lakehouse` even though their management would be separated (as in Fig. 9). On the other hand, if Raw Sales should be detached from the following part of the pipeline, then it would implemented using `Azure Blob Storage` rather than `Lakehouse` (being a data lake that is not eventually connected to a data warehouse, we correctly discard `Lakehouse` as a possible implementation).

Both companies acknowledged that selecting services for a project is highly complex and that staying up to date about service provider offerings requires considerable time and effort. Overall the experts were enthusiastic about the approach and highlighted the following strengths: (i) it can support and train young designers for the creation of blueprints, (ii) it can remove bias in the selection of services (e.g., consultants mainly working with a hyperscaler could tend to prefer services that hyperscaler), and (iii) it is an objective approach to compare different blueprints. Although the blueprints are not perfectly overlapping, experts claim that our blueprints are correct from a technical and technological point of view. However, what could be improved in our approach is the integration of more organizational, political, or even familiarity constraints that could bias the service selection. Our approach partially addresses these issues during the augmentation of the matched graph (e.g., with properties isAkin, mandatory, and preferred) but could benefit by allowing further expressiveness.

**Table 3**
Breakdown of performance (in seconds) by increasing the nodes of the DFD.

| $\|N^D\|$ | Match | Augment | Select | Total |
|---|---|---|---|---|
| 10 | 0.22 | $4 \cdot 10^{-5}$ | 0.17 | 0.38 |
| 50 | 0.87 | $4 \cdot 10^{-5}$ | 0.26 | 1.13 |
| 250 | 7.23 | $5 \cdot 10^{-5}$ | 0.53 | 7.76 |

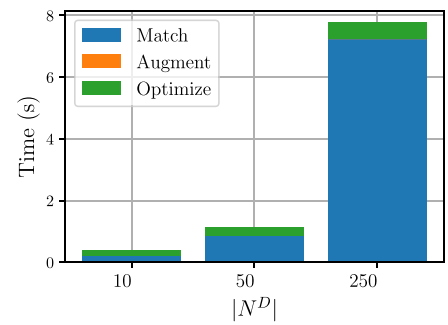


**Fig. 11.** Performance by increasing the size of the DFD.

### 7.2. Efficiency

We test how our approach scales up to bigger cloud ecosystems and data pipelines, both translating into bigger service and DFD graphs with more nodes and relationships.

To do so, we implement additional synthetic pipelines (i.e., DFDs with increasing nodes $|N^D|$) that share the service ecosystem and the taxonomy of tags. DFDs are created using a tree structure that is randomly expanded until a given number of nodes is reached. The shared service ecosystem contains 200 services with a compatibility rate of 95% (that is, each service is compatible with 95% of the others). Each service is enriched with a pair of tags randomly extracted from the shared taxonomy (using a uniform distribution). Finally, each node in the DFD is enriched with a pair of tags selected from the set of tag pairs previously assigned to the services within the service ecosystem. This ensures that every node in the DFD is implemented by at least one service.

To assess the scalability of the approach, the synthetic pipelines differ by a factor of 5x in terms of the number of nodes composing each DFD, starting with DFDs of 10 nodes (as for the real pipelines described in Section 7.1) up to 250 nodes.[4] For each number of nodes $N^D \in [10, 50, 250]$ we randomly generate 10 DFD graphs and Table 3 and Fig. 11 summarize the average results. Overall, our approach scales well to scenarios involving 200 services and 250 data processes/repositories in the DFD, with a computational time of less than 10 s. The time necessary for augmentation and pruning is negligible, whereas matching dominates the total time (93% for 250 nodes). Matching could be improved by further optimizing GraphDB since the match is automatically solved by the reasoner.

[4] We use DFDs of 250 nodes to stress our system although we believe that they are hardly manageable in real applications.

## 8. Related work

Understanding which "building blocks" should be part of a data platform is nontrivial. Several functional architectures (system representations that focus on its functions and their interactions) have been proposed. NIST designed the Big Data Reference Architecture [2] which comprised vendor-, technology- and infrastructure-independent logical functional components that are necessary for managing and processing big data (data provider, data consumer, system orchestrator, big data application provider, and big data framework provider). Lambda [3] is an architecture designed to handle massive quantities of data by taking advantage of both batch and stream-processing methods. Kappa [4] overcomes some limitations (e.g., "redundant" batch and streaming implementations) of Lambda by using a pure streaming approach with a single code base. While these architectures define functional components necessary for (big) data platforms, they are abstract and do not address the problem of selecting the optimal services necessary for designing, implementing, and deploying working data platforms.

Cloud service providers such as Amazon [13], Google [14], and Microsoft [15] provide ecosystems of independent and interoperable services. While ecosystems enable easy deployment of data pipelines, choosing the *optimal* services is hard: (i) each provider offers different services, (ii) a shared categorization and organization of services is missing, and (iii) it is hard for the users to understand which services are needed based on their data-driven processes [16]. Some multi-objective optimization techniques have been proposed, where users express requirements and preferences (e.g., minimal QoS) about single services [17–19], cloud deployment models (public and private) [20], and service providers [21,22]. Finally, domain languages and ontologies have been introduced (e.g., [23]) to enable some service composition (e.g., [24]).

With respect to these works, the novelties of our approach are (i) the introduction of a methodology to enable data platform design, (ii) a computer-aided approach to support designers in their choices, and (iii) process-driven optimization to select the optimal blueprint out of many data flows, even from multiple stakeholders.

## 9. Conclusion and future works

The design of data platforms is a nontrivial task. In this paper, we have introduced a methodology to aid designers in selecting the optimal services (out of a service ecosystem) supporting data-driven processes from multiple stakeholders (e.g., partners in a research project), and we have addressed such selection as a facility location optimization problem. Our approach can match and select single services as well as design patterns (e.g., Lakehouse to replace both data lakes and warehouses), it supports additional constraints (e.g., mandatory and affinity), and the produced blueprints have been successfully compared with the ones recommended by expert designers.

We believe that the expressiveness of the approach can still be improved as follows. *Expressiveness*: introduce more advanced constraints to support organizational and political factors in the design of the blueprints. *Resource provisioning*: additionally to selecting the services, a complete approach should also consider how many instances of a service are required (to do so, a cost model should be studied). *Metadata integration*: while catalog and meta-data management services do not directly introduce functionalities for data transformation and exploitation, the design should also recommend services helping in the management of the platform itself [25].

## CRediT authorship contribution statement

**Matteo Francia:** Writing – original draft, Writing - Review & Editing, Validation, Software, Methodology, Conceptualization. **Matteo Golfarelli:** Writing – original draft, Writing - Review & Editing, Supervision, Methodology, Conceptualization. **Manuele Pasini:** Writing – original draft, Writing - Review & Editing, Validation, Software, Methodology.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## References


[1] M. Zaharia, A. Ghodsi, R. Xin, M. Armbrust, Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, in: CIDR, www.cidrdb.org, 2021, p. 8, URL http://cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf.

[2] W. Chang, N. Grady, N.-P. NIST, NIST big data interoperability framework: Volume 2, big data taxonomies [version 2], 2018, http://dx.doi.org/10.6028/NIST.SP.1500-2r1.

[3] M. Kiran, P. Murphy, I. Monga, J. Dugan, S.S. Baveja, Lambda architecture for cost-effective batch and speed big data processing, in: BigData, IEEE, Santa Clara, CA, USA, 2015, pp. 2785–2792, http://dx.doi.org/10.1109/BIGDATA.2015.7364082.

[4] J. Kreps, Questioning the Lambda Architecture, vol. 2, O'Reilly, 2014, p. 10.

[5] M. Francia, M. Golfarelli, M. Pasini, Towards a process-driven design of data platforms, in: E. Gallinucci, M. Lissandrini (Eds.), Proceedings of the 26th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP 2024), Paestum, Italy, March 25, 2024, in: CEUR Workshop Proceedings, vol. 3653, CEUR-WS.org, 2024, pp. 28–35, URL https://ceur-ws.org/Vol-3653/paper3.pdf.

[6] T. DeMarco, Structured Analysis and System Specification, Yourdon Press, 1978, p. 366.

[7] Earth observation guide, 2020, https://business.esa.int/newcomers-earth-observation-guide. (Accessed 26 July 2024).

[8] Agritech, 2024, https://agritechcenter.it/. (Accessed 26 July 2024).

[9] M. Francia, J. Giovanelli, M. Golfarelli, Multi-sensor profiling for precision soil-moisture monitoring, Comput. Electron. Agric. 197 (2022) 106924, http://dx.doi.org/10.1016/j.compag.2022.106924.

[10] E. Baldi, M. Quartieri, G. Larocca, M. Golfarelli, M. Francia, J. Giovanelli, E. Xylogiannis, M. Toselli, Smart irrigation system for precision water management: effect on yield and fruit quality of yellow fleshed kiwifruit in northern Italy, in: ECPA, Wageningen Academic Publishers, 2023, pp. 59–66, http://dx.doi.org/10.3920/978-90-8686-947-3_5.

[11] J. Anuradha, et al., A brief introduction on big data 5vs characteristics and hadoop technology, Procedia Comput. Sci. 48 (2015) 319–324, http://dx.doi.org/10.1016/j.procs.2015.04.188.

[12] DataForm, 2024, https://cloud.google.com/dataform. (Accessed 26 July 2024).

[13] Amazon web services, 2024, https://aws.amazon.com/. (Accessed 26 July 2024).

[14] Google cloud platform, 2024, https://cloud.google.com/. (Accessed 26 July 2024).

[15] Microsoft azure, 2024, https://azure.microsoft.com/en-us. (Accessed 26 July 2024).

[16] L. Sun, H. Dong, F.K. Hussain, O.K. Hussain, E. Chang, Cloud service selection: State-of-the-art and future research directions, J. Netw. Comput. Appl. 45 (2014) 134–150, http://dx.doi.org/10.1016/J.JNCA.2014.07.019.

[17] E. Cavalcante, F. Lopes, T.V. Batista, N. Cacho, F.C. Delicato, P.F. Pires, Cloud integrator: Building value-added services on the cloud, in: NCCA, IEEE, Toulouse, France, 2011, pp. 135–142, http://dx.doi.org/10.1109/NCCA.2011.29.

[18] M.H. Nejat, H. Motameni, H. Vahdat-Nejad, B. Barzegar, Efficient cloud service ranking based on uncertain user requirements, Clust. Comput. 25 (1) (2022) 485–502, http://dx.doi.org/10.1007/S10586-021-03418-W.

[19] L. Lu, Y. Yuan, A novel TOPSIS evaluation scheme for cloud service trustworthiness combining objective and subjective aspects, J. Syst. Softw. 143 (2018) 71–86, http://dx.doi.org/10.1016/J.JSS.2018.05.004.

[20] R. Garg, MCDM-based parametric selection of cloud deployment models for an academic organization, IEEE Trans. Cloud Comput. 10 (2) (2022) 863–871, http://dx.doi.org/10.1109/TCC.2020.2980534.

[21] M.B. Kar, R. Krishankumar, D. Pamucar, S. Kar, A decision framework with nonlinear preferences and unknown weight information for cloud vendor selection, Expert Syst. Appl. 213 (Part) (2023) 118982, http://dx.doi.org/10.1016/J.ESWA.2022.118982.

[22] A. Tomar, R.R. Kumar, I. Gupta, Decision making for cloud service selection: a novel and hybrid MCDM approach, Clust. Comput. 26 (6) (2023) 3869–3887, http://dx.doi.org/10.1007/S10586-022-03793-Y.

[23] M. Noura, M. Gaedke, WoTDL: Web of things description language for automatic composition, in: Int. Conf. on Web Intelligence, ACM, Thessaloniki, Greece, 2019, pp. 413–417, http://dx.doi.org/10.1145/3350546.3352558.

[24] H.B. Mahfoudh, A. Caselli, G.D. Serugendo, Learning-based coordination model for on-the-fly self-composing services using semantic matching, J. Sens. Actuator Netw. 10 (1) (2021) 5, http://dx.doi.org/10.3390/JSAN10010005.

[25] M. Francia, E. Gallinucci, M. Golfarelli, A.G. Leoni, S. Rizzi, N. Santolini, Making data platforms smarter with MOSES, Future Gener. Comput. Syst. 125 (2021) 299–313, http://dx.doi.org/10.1016/j.future.2021.06.031.